

Конвертер представлений XML— JsonML для редактора "богатого текста"

Конвертер XML-JsonML предназначен для преобразования текстовых строк специальной структуры, которые являются различными представлениями текста для БТР (rich text editor, редактор "богатого текста"), из формата XML в формат JsonML и обратно. Конвертер реализован на C++, а для удобства использования в прикладном коде сделаны два БЛ-метода.

JsonML Richtext представление "богатого текста" XML описано [здесь](#).

Конвертер XML → JsonML

Класс

```
sbis::jsonxml::XmlJsonConverter
```

представляет собой конвертер из XML в JsonML.

Конструктор класса

```
XmlJsonConverter::XmlJsonConverter( XJErrorLevel error_tolerance =  
XJErrorLevel::lError );
```

принимает в качестве параметра уровень ошибок, считающихся критичными (варианты: lFatal, lError, lWarning, по умолчанию используется lError).

Метод

```
bool XmlJsonConverter::SetOption( Option option, Int32 value );
```

позволяет задавать дополнительные опции конвертера, влияющие на обработку текста.

Название опции	Описание	Возможные значения
XmlJsonConverter::oUseStrictXml	Распознавать только канонический XML.	1 – конвертер распознаёт только канонический XML. Никаких попыток исправить ошибки с помощью сторонней библиотеки Tidy. 0 (по умолчанию) – если конвертеру на вход передан невалидный XML, тогда для исправления ошибок используется сторонняя библиотек Tidy.
XmlJsonConverter::oTreatUnknownTags	Как обрабатывать неизвестные теги. Список известных тегов приведён здесь .	0 – оставить как есть. Вероятно, что в таком случае XML будет расценен как невалидный. 1 (по умолчанию) – вырезать неизвестные теги, но содержимое оставить;

		• 2 – экранировать неизвестные теги; • 3 – вырезать неизвестные теги вместе с содержимым. before <invalid attr="1">inside</invalid> after before after
XmlJsonConverter::oTrimSpaces	Удалить незначащие символы (пробелы, табуляции, символы переноса строки) в начале и в конце строки перед обработкой.	• 1 – да; • 0 (по умолчанию) – нет;
XmlJsonConverter::oEscapeWholeText	Рассматривать весь текст как plain text?	• 1 – да; • 0 (по умолчанию) – нет;
XmlJsonConverter::oNoPlainText	Расценивать строку как XML/HTML, и не рассматривать её как plain text.	• 1 – да; • 0 (по умолчанию) – нет;

Convert(). Преобразовать строку из формата XML в формат JsonML

Параметры	Описание
Вход	Строка формата XML.
Выход	Строка в формате JsonML. Если входная строка не валидна, возвращается пустая строка.

Примеры

• C++:

```

1 #include <sbis-xml-json-converter/xml_json.hpp>
2
3 XmlJsonConverter converter;
4 String richtext_source = LR"===(<span style="color:
  blue;">Текст</span>)===";
5 String result_json = converter.Convert( richtext_source );
6 // result_json is LR"===([["span", { "style": "color: blue;" },
  "Текст"]])==="
```

• RPC вызов из C++:

```

1 String xml = LR"===(<span style="color: blue;">Текст</span>)===";
2 blcore::EndPoint ep( L"Converter" );
3 blcore::Object obj( L"XmlJson", ep );
4 String res = obj.Invoke< String >( L"Convert", xml );
5 //res is LR"===([["span", {"style": "color: blue;"}, "Текст"]])==="
```

• Python:

```

1 import xml_json
2
3 xjc = xml_json.XmlJsonConverter( xml_json.XJErrorLevel.lError )
4 xml = '<span style="color: blue;">Текст</span>'
```

```

5 | json = xjc.Convert( xml )
6 | # json is '[[{"span", { "style": "color: blue;" }, "Текст"}]]'

```

ConvertBatch(). Преобразовать строки из формата XML в формат JsonML

Параметры	Описание
Вход	Массив строк формата XML.
Выход	Массив строк в формате JsonML. Если какая-то из входных строк не валидна, на выходе ей соответствует пустая строка.

Примеры

- C++:

```

1 | #include <sbis-xml-json-converter/xml_json.hpp>
2 |
3 | XmlJsonConverter converter;
4 | ArrayString xml( 2 );
5 | xml[ 0 ] = LR"===(<span style="color: blue;">Текст</span>===";
6 | xml[ 1 ] = LR"===(<div>test</div>===";
7 | ArrayString result = converter.ConvertBatch( xml );
8 | //result[ 0 ] is LR"===([["span",{ "style": "color: blue;" },
  | "Текст"]])===
9 | //result[ 1 ] is LR"===([["div", "test"]])===

```

- RPC вызов из C++:

```

1 | ArrayString xml( 2 );
2 | xml[ 0 ] = LR"===(<span style="color: blue;">Текст</span>===";
3 | xml[ 1 ] = LR"===(<div>test</div>===";
4 | blcore::EndPoint ep( L"Converter" );
5 | blcore::Object obj( L"XmlJson", ep );
6 | ArrayString result = obj.Invoke< ArrayString >( L"ConvertBatch", xml
  | );
7 | // result[ 0 ] is LR"===([["span", { "style": "color: blue;" },
  | "Текст"]])===
8 | // result[ 1 ] is LR"===([["div", "test"]])===

```

- Python:

```

1 | import xml_json
2 |
3 | xjc = xml_json.XmlJsonConverter( xml_json.XJErrorLevel.lError )
4 | arr_xml = [ '<div>sometext</div>', '<span style="color:
  | blue;">Текст</span>' ]
5 | arr_json = xjc.ConvertBatch( arr_xml )
6 | # arr_json is [ '[[{"div", "sometext"}]]', '[[{"span", { "style":
  | "color: blue;" }, "Текст"}]]' ]

```

TryConvert(). Преобразовать входную строку в формат JsonML

Параметры	Описание
Вход	Строка неизвестного формата, XML или JsonML.

Примеры

- C++:

```

1 #include <sbis-xml-json-converter/xml_json.hpp>
2
3 XmlJsonConverter converter;
4 String in_xml = LR"===(<span style="color: blue;">Текст</span>)===";
5 String in_json = LR"===([["span", { "style": "color: blue;" },
  "Текст"]])===";
6 String result_1 = converter.TryConvert( in_xml );
7 String result_2 = converter.TryConvert( in_json );
8 // result_1 is LR"===([["span", { "style": "color: blue;" },
  "Текст"]])===";
9 // result_2 is LR"===([["span", { "style": "color: blue;" },
  "Текст"]])===";

```

- RPC вызов из C++:

```

1 String xml = LR"===(<span style="color: blue;">Текст</span>)===";
2 String json=LR"===([["span", { "style": "color: blue;" },
  "Текст"]])===";
3 blcore::EndPoint ep( L"Converter" );
4 blcore::Object obj( L"XmlJson", ep );
5 String res_1 = obj.Invoke<String>( L"TryConvert", xml );
6 String res_2 = obj.Invoke<String>( L"TryConvert", json );
7 // res_1 is LR"===([["span", { "style": "color: blue;" },
  "Текст"]])===";
8 // res_2 is LR"===([["span", { "style": "color: blue;" },
  "Текст"]])===";

```

- Python:

```

1 import xml_json
2
3 xjc = xml_json.XmlJsonConverter( xml_json.XJErrorLevel.lError )
4 xml = '<span style="color: blue;">Текст</span>'
5 json = '[[["span", { "style": "color: blue;" }, "Текст"]]]'
6 res_1 = xjc.TryConvert( xml )
7 res_2 = xjc.TryConvert( json )
8 # res_1 is '[[["span", { "style": "color: blue;" }, "Текст"]]]'
9 # res_2 is '[[["span", { "style": "color: blue;" }, "Текст"]]]'

```

TryConvertBatch(). Преобразовать входные строки в формат JsonML

Параметры	Описание
Вход	Массив строк неизвестного формата, XML или JsonML.
Выход	Если какая-то из входных строк не валидна, на выходе ей соответствует пустая строка. Если на входе валидный JsonML – выдать без изменений. Если на входе валидный XML – преобразовать в JsonML. Иначе выдать пустую строку.

Вызовы производятся аналогично **ConvertBatch**.

Конвертер JsonML → XML

Класс

```
sbis::jsonxml::JsonXmlConverter
```

представляет собой конвертер из JsonML в XML.

Конструктор

```
JsonXmlConverter::JsonXmlConverter();
```

Конвертер JsonML в XML реализован на основе парсера, описанного [здесь](#).

Convert(). Преобразовать строку из формата JsonML в формат XML

Параметры	Описание
Вход	Строка в формате JsonML.
Выход	Строка в формате XML. Если входная строка не валидна, возвращается пустая строка.

Примеры

- C++:

```
1 #include <sbis-xml-json-converter/json_xml.hpp>
2
3 JsonXmlConverter converter;
4 String json_source = LR"===([["span", "sometext"]])===";
5 String result_xml = converter.Convert( json_source );
6 // result_json is LR"===(<span>sometext</span>)==="
```

- RPC вызов из C++:

```
1 String json=LR"===([["span", "sometext"]])===";
2 blcore::EndPoint ep( L"Converter" );
3 blcore::Object obj( L"JsonXml", ep );
4 String res = obj.Invoke< String >( L"Convert", json );
5 // res is LR"===(<span>sometext</span>)==="
```

- Python:

```
1 import xml_json
2
3 jxc = xml_json.JsonXmlConverter()
4 json = '[[["span", { "style": "color: blue;" }, "Текст"]]]'
5 xml = jxc.Convert( json )
6 # xml is '<span style="color: blue;">Текст</span>' )
```

ConvertBatch(). Преобразовать строки из формата JsonML в формат XML

--	--

Параметры	Описание
Вход	Массив строк в формате JsonML.
Выход	Массив строк в формате XML. Если какая-то из входных строк не валидна, на выходе ей соответствует пустая строка.

Примеры

- C++:

```

1 #include<sbis-xml-json-converter/json_xml.hpp>
2
3 JsonXmlConverter converter;
4 StringArray json( 2 );
5 json[ 0 ] = LR"===([["span", "sometext"]])===";
6 json[ 1 ] = LR"===([["span", { "style": "color: blue;" },
7   "Текст"]])===";
8 StringArray result = converter.ConvertBatch( json );
9 // result[ 0 ] is LR"===(<span>sometext</span>)==="
10 // result[ 1 ] is LR"===(<span style="color: blue;">Текст</span>)==="

```

- RPC вызов из C++:

```

1 StringArray json( 2 );
2 json[ 0 ] = LR"===([["span", "sometext"]])===";
3 json[ 1 ] = LR"===([["span", { "style": "color: blue;" },
4   "Текст"]])===";
5 blcore::EndPoint ep( L"Converter" );
6 blcore::Object obj( L"JsonXml", ep );
7 StringArray res = obj.Invoke< StringArray >( L"ConvertBatch", json );
8 // res[ 0 ] is LR"===(<div>sometext</div>)==="
9 // res[ 1 ] is LR"===(<span style="color: blue;">Текст</span>)==="

```

- Python:

```

1 import xml_json
2
3 jxc = xml_json.JsonXmlConverter()
4 json = [ '["span", { "style": "color: blue;" }, "Текст"]',
5   '["span", "sometext"]' ]
6 xml = jxc.ConvertBatch( json )
7 # xml is [ '<span style="color: blue;">Текст</span>',
8   '<span>sometext</span>' ]

```

WrapLinks() Выделить ссылки в Json строке

Найденные в тексте ссылки оборачиваются в теги ["a", { "href": ... }, ...]. Если ссылки уже выделены (находятся внутри тега "a"), они не выделяются.

Параметры	Описание
Вход	Строка в формате Json.
Выход	Строка в формате Json с обрамленными ссылками.

Примеры

- C++:

```

1 #include <sbis-xml-json-converter/json_xml.hpp>

```

```

2
3 JsonSerializer converter;
4 String source_json = LR"===([["p", { "version": "2" }, "Link
http:\\\\www.google.com\\ / text"]])===";
5 String result_json = converter.WrapLinks( source_json );
6 // result_json is LR"===([["p", { "version": "2" }, "Link ", ["a", {
"href": "http:\\\\www.google.com\\ /", "target": "_blank" },
"http:\\\\www.google.com\\ /"], " text"]])===";

```

- Python:

```

1 import xml_json
2
3 jxc = xml_json.JsonXmlConverter()
4 source_json = '["p", { "version": "2" }, "Link
http:\\\\www.google.com\\ / text"]]'
5 result_json = jxc.WrapLinks( source_json )
6 # result_json is '["p", { "version": "2" }, "Link ", ["a", { "href":
"http:\\\\www.google.com\\ /", "target": "_blank" },
"http:\\\\www.google.com\\ /"], " text"]]'

```

WrapLinksBatch() Выделить ссылки в массиве Json строк

Найденные в тексте ссылки оборачиваются в теги ["a", { "href": ... }, ...]. Если ссылки уже выделены (находятся внутри тега "a"), они не выделяются.

Параметры	Описание
Вход	Массив Json строк.
Выход	Массив Json строк с обрамленными ссылками.

Примеры

- C++:

```

1 #include<sbis-xml-json-converter/json_xml.hpp>
2
3 JsonSerializer converter;
4 StringArray source_json( 2 );
5 source_json[ 0 ] = LR"===([["p", { "version": "2" }, "Link
http:\\\\www.google.com \\ / text"]])===";
6 source_json[ 1 ] = LR"===([["p", { "version": "2" }, "Link ", ["a", {
"href": "http:\\\\www.google.com \\ /", "target": "_blank" },
"http:\\\\www.google.com \\ /"], " text"]])===";
7 StringArray result_json = converter.WrapLinksBatch( source_json );
8 // result_json[ 0 ] is LR"===([["p", { "version": "2" }, "Link ",
["a", { "href": "http:\\\\www.google.com \\ /", "target": "_blank" },
"http:\\\\www.google.com \\ /"], " text"]])===";
9 // result_json[ 1 ] is LR"===([["p", { "version": "2" }, "Link ",
["a", { "href": "http:\\\\www.google.com \\ /", "target": "_blank" },
"http:\\\\www.google.com \\ /"], " text"]])===";

```

- Python:

```

1 #import xml_json
2
3 jxc = xml_json.JsonXmlConverter()
4 source_json = [ '["p", { "version": "2" }, "Link
http:\\\\www.google.com\\ / text"]]', '["p", { "version": "2" }, "Link

```

```

    ", [ "a", { "href": "http:\\\\www.google.com\\/", "target": "_blank" },
    "http:\\\\www.google.com\\/"], " text"]]' ]
5 result_json = jxc.WrapLinksBatch( source_json )
6 # result_json is [ '[[ "p", { "version": "2" }, "Link ", [ "a", {
    "href": "http:\\\\www.google.com\\/", "target": "_blank" },
    "http:\\\\www.google.com\\/"], " text"]]', '[[ "p", { "version": "2" },
    "Link ", [ "a", { "href": "http:\\\\www.google.com\\/", "target":
    "_blank" }, "http:\\\\www.google.com\\/"], " text"]]' ]

```

ExtractText(). Извлечь введенный пользователем текст без разметки

Параметры	Описание
Вход	Строка в формате JsonML.
Выход	Текст в формате XML.

Примеры

- C++:

```

1 #include <sbis-xml-json-converter/json_xml.hpp>
2
3 JsonXmlConverter converter;
4 String json = LR"==(["span", "sometext"])==";
5 String result = converter.ExtractText( json );
6 //result is "sometext"

```

- RPC вызов из C++:

```

1 String json = LR"==(["span", "sometext"])==";
2 blcore::EndPoint ep( L"Converter" );
3 blcore::Object obj( L"JsonXml", ep );
4 String res = obj.Invoke< String >( L"ExtractText", json );
5 //res is "sometext"

```

- Python:

```

1 import xml_json
2 jxc = xml_json.JsonXmlConverter()
3 json = '[[ "span", { "style": "color: blue;" }, "Текст"]]'
4 text = jxc.ExtractText( json )
5 # text is 'Текст'

```

ExtractTextBatch(). Извлекает введенный пользователем текст без разметки

Параметры	Описание
Вход	Массив строк в формате JsonML.
Выход	Массив строк с текстом в формате XML, извлеченным из входных строк.

Примеры

- C++:

```

1 #include <sbis-xml-json-converter/json_xml.hpp>

```



```

2
3 JsonSerializer converter;
4 StringArray json( 2 );
5 json[ 0 ] = LR"==(["span", "sometext"])==";
6 json[ 1 ] = LR"==(["span", { "style": "color: blue;" },
  "Текст"])==";
7 StringArray result = converter.ExtractTextBatch(json);
8 // result[ 0 ] is "sometext"
9 // result[ 1 ] is "Текст"

```

- RPC вызов из C++:

```

1 StringArray json( 2 );
2 json[ 0 ] = LR"==(["span", "sometext"])==";
3 json[ 1 ] = LR"==(["span", { "style": "color: blue;" },
  "Текст"])==";
4 blcore::EndPoint ep( L"Converter" );
5 blcore::Object obj( L"JsonXml", ep );
6 StringArray res = obj.Invoke< StringArray >( L"ExtractTextBatch",
  json );
7 // res[ 0 ] is "sometext"
8 // res[ 1 ] is "Текст"

```

- Python:

```

1 import xml_json
2
3 jxc = xml_json.JsonXmlConverter()
4 json = [ '["span", { "style": "color: blue;" }, "Текст"]',
  '["span", "sometext"]' ]
5 xml = jxc.ExtractTextBatch( json )
6 # xml is [ 'Текст', 'sometext' ]

```

Подключение конвертера в прикладной проект

- Для работы с конвертером из исходного кода C++:
 - подключите сервисные модули [XmlJsonConverter.s3mod](#) в состав вашего сервиса;
 - подключите библиотеку [sbis-xml-json-converter](#);
- Для работы с конвертером из исходного кода Python:
 - подключите сервисные модули [XmlJsonConverter.s3mod](#) и [XmlJsonConverter-Py.s3mod](#) в состав вашего сервиса;
 - импортируйте модуль `xml_json`;